

MLDL Experiment 4

Aim: Implement K-Nearest Neighbors (KNN) and evaluate model performance

1. Dataset Source

Dataset Name: Pima Indians Diabetes Database

Source: [Kaggle - Pima Indians Diabetes Database](#)

2. Dataset Description

This dataset is originally from the National Institute of Diabetes and Digestive and Kidney Diseases. The objective is to predict whether a patient has diabetes based on diagnostic measurements.

- **Size:** 768 instances, 9 columns.
- **Target Variable:** **Outcome** (0 = No Diabetes, 1 = Diabetes).
- **Key Features:** All features are numerical inputs.
 - **Pregnancies:** Number of times pregnant.
 - **Glucose:** Plasma glucose concentration.
 - **BloodPressure:** Diastolic blood pressure (mm Hg).
 - **SkinThickness:** Triceps skin fold thickness (mm).
 - **Insulin:** 2-Hour serum insulin (mu U/ml).
 - **BMI:** Body mass index.
 - **DiabetesPedigreeFunction:** Diabetes pedigree function.
 - **Age:** Age (years).

3. Mathematical Formulation of the Algorithm

KNN is a **lazy learner** that classifies a new data point based on the majority class of its k nearest neighbors in the feature space.

1. Distance Metric (Euclidean Distance):

To find the "nearest" neighbors, KNN calculates the distance between the query point x and a training point y . The most common metric is Euclidean distance:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Where:

- x_i and y_i are the values of the i -th feature for instances x and y .
- n is the total number of features.

2. Classification Decision:

Once the k nearest neighbors are identified, the algorithm assigns the class label by **Majority Voting**:

$$\hat{y} = \text{mode}\{y_1, y_2, \dots, y_k\}$$

Where y_i is the class label of the i -th neighbor.

4. Algorithm Limitations

- **Computationally Expensive:** KNN is a "lazy" algorithm; it does not learn a model during training. Instead, it stores all data and calculates distances for every single prediction, making it slow with large datasets.
- **Sensitivity to Scale:** Features with larger ranges (e.g., Insulin 0-800) will dominate the distance calculation over small ranges (e.g., Pregnancies 0-17). **Feature Scaling** is mandatory.
- **Curse of Dimensionality:** As the number of features increases, the "distance" between points becomes less meaningful, degrading performance.

5. Methodology

1. **Data Loading:** Load `diabetes.csv`.
2. **Preprocessing:**
 - Split data into Features (X) and Target (y).
 - **Scaling:** Apply `StandardScaler` to normalize all features so they contribute equally to the distance metric.
3. **Train-Test Split:** Split data (80% Train, 20% Test).
4. **Model Training:** Initialize KNN (starting with $K=5$) and fit on training data.
5. **Prediction:** Predict labels for the test set.
6. **Evaluation:** Calculate Accuracy and Confusion Matrix.
7. **Hyperparameter Tuning:** Use the **Elbow Method** (iterate through $K=1$ to 40) to find the optimal K value that minimizes error.

6. Performance Analysis

The KNN model was evaluated on a test set of **154 instances** (20% of the dataset).

A. Initial Model Evaluation (K=5)

- **Accuracy:** The initial model achieved an accuracy of **69.48%**.
- **Confusion Matrix Analysis:**
 - **True Negatives (No Diabetes):** 79
 - **True Positives (Diabetes):** 28
 - **False Negatives (Missed Detection):** 27
 - **Observation:** The model struggled significantly with Class 1 (Diabetes), misclassifying nearly as many positive cases (27) as it correctly identified (28). This resulted in a low Recall score of 0.51 for the diabetes class.

B. Hyperparameter Tuning (Elbow Method)

- The **Error Rate vs. K Value** graph demonstrates that as K increased from 1 to 20, the error rate generally decreased and stabilized.
 - **Optimal K:** The lowest error rate was found at **K=21**.
 - **Improvement:** Tuning the hyperparameter improved the model accuracy from **69.48%** to **76.62%**.
-

Python Code

```
#import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 2. Load Dataset
try:
    df = pd.read_csv('diabetes.csv')
    print("Dataset Loaded Successfully.")
    print(df.head())
except FileNotFoundError:
```

```

print("Error: 'diabetes.csv' not found. Please upload the dataset.")

# 3. Preprocessing (CRUCIAL for KNN)
X = df.drop('Outcome', axis=1)
y = df['Outcome']

# Split Data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature Scaling (Standardization)
# KNN calculates distances, so features must be on the same scale.
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("\nData Scaled successfully.")

# 4. Train KNN Model (Initial K=5)
k = 5
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train, y_train)

# Predict
y_pred = knn.predict(X_test)

# 5. Evaluate Performance
accuracy = accuracy_score(y_test, y_pred)
print(f"\n--- KNN Performance (K={k}) ---")
print(f"Accuracy: {accuracy:.4f}")
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Purples')
plt.title(f"Confusion Matrix (K={k})")
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# 6. Hyperparameter Tuning (Finding Best K)
print("\n--- Hyperparameter Tuning (Elbow Method) ---")

```

```

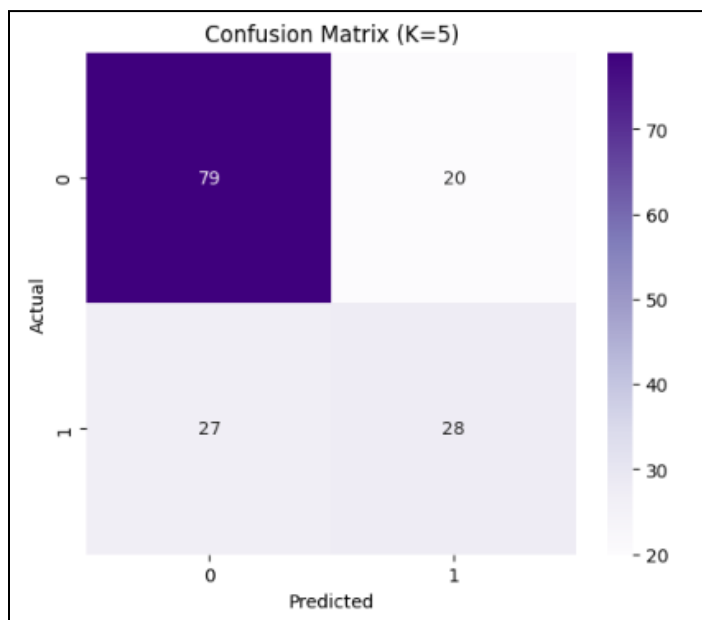
error_rate = []
accuracy_list = []

# Will take some time
for i in range(1, 40):
    knn_i = KNeighborsClassifier(n_neighbors=i)
    knn_i.fit(X_train, y_train)
    pred_i = knn_i.predict(X_test)
    error_rate.append(np.mean(pred_i != y_test))
    accuracy_list.append(accuracy_score(y_test, pred_i))

# Plot Error Rate vs K
plt.figure(figsize=(10, 6))
plt.plot(range(1, 40), error_rate, color='blue', linestyle='dashed', marker='o',
         markerfacecolor='red', markersize=10)
plt.title('Error Rate vs. K Value')
plt.xlabel('K')
plt.ylabel('Error Rate')
plt.show()

# Find the K with the maximum accuracy (min error)
best_k = accuracy_list.index(max(accuracy_list)) + 1
print(f'Optimal K value found: {best_k}')
print(f'Maximum Accuracy at K={best_k}: {max(accuracy_list):.4f}')

```



Dataset Loaded Successfully.

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI \
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

Data Scaled successfully.

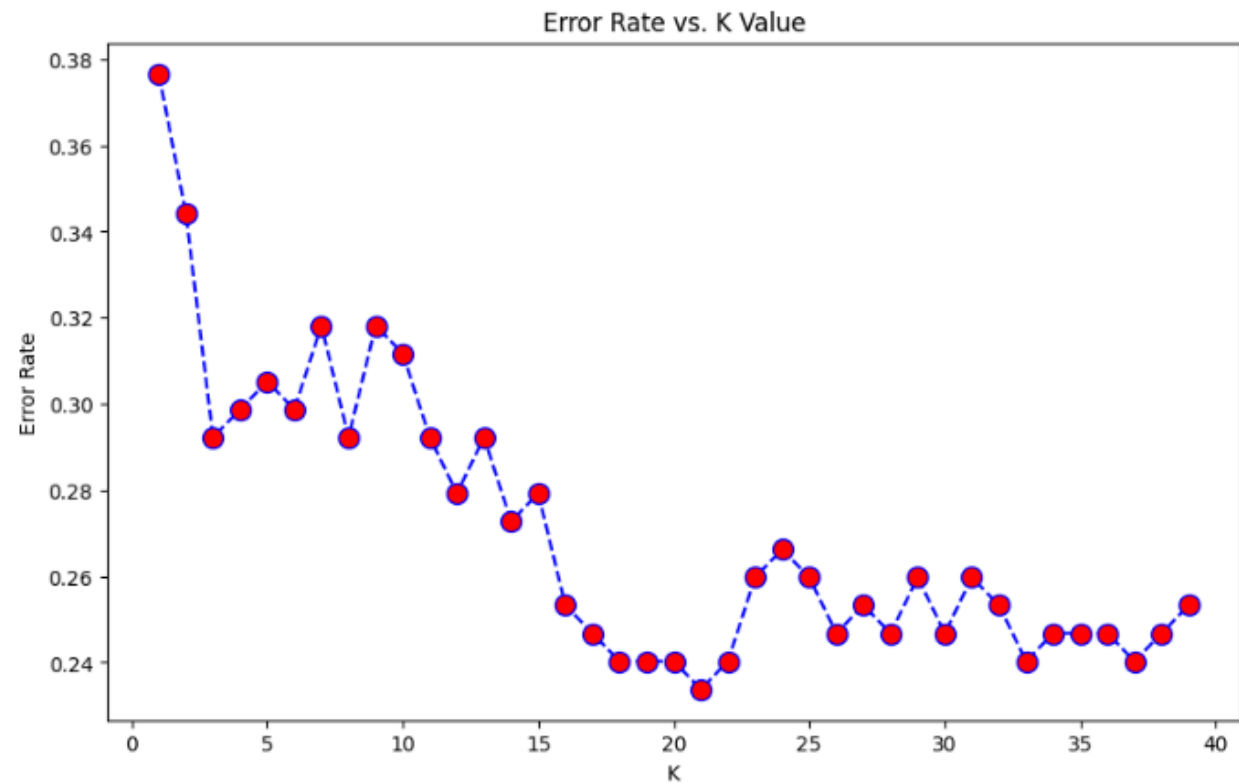
--- KNN Performance (K=5) ---

Accuracy: 0.6948

Classification Report:

	precision	recall	f1-score	support
0	0.75	0.80	0.77	99
1	0.58	0.51	0.54	55
accuracy			0.69	154
macro avg	0.66	0.65	0.66	154
weighted avg	0.69	0.69	0.69	154

--- Hyperparameter Tuning (Elbow Method) ---



Optimal K value found: 21

Maximum Accuracy at K=21: 0.7662

Conclusion

In this experiment, we successfully implemented the K-Nearest Neighbors (KNN) algorithm to predict the onset of diabetes.

1. **Impact of Tuning:** The experiment demonstrated that KNN is highly sensitive to the choice of K. By increasing the number of neighbors from the default (K=5) to the optimal value (K=21), we achieved a significant accuracy boost of approximately **7.14%** (from 69.5% to 76.6%).
2. **Bias-Variance Tradeoff:** The initial lower K resulted in overfitting (high variance), capturing noise in the data which led to poor generalization on the test set. Increasing K smoothed the decision boundary, reducing the error rate.
3. **Data Sensitivity:** The substantial number of False Negatives in the initial model highlights the importance of choosing the right metric (Recall vs. Precision) for medical diagnoses, where missing a positive case is critical.

Final Verdict: KNN proved to be a capable classifier for this dataset, provided that feature scaling is applied and the neighbor count (K) is properly optimized.